DESENVOLVIMENTO DE API BACK-END · ENGENHARIA DE SOFTWARE · 6º PERÍODO

Semana 14
Integração contínua, deploy e versionamento

Apostila de estudo: teoria, exemplos, construção com IA e exercícios

Até aqui a API rodou na sua máquina e no Compose. Nesta semana ela sai do localhost. Você vai entender o caminho que uma mudança percorre do commit até o usuário, por que uma máquina limpa deve provar cada push, como escolher onde hospedar e como trocar a versão em execução sem derrubar ninguém. Por fim, como evoluir o contrato público sem quebrar clientes que você não controla. Integração contínua aparece aqui em nível introdutório: o tema tem disciplina própria no 7º período.

Conteúdo

1. Objetivos da semana
2. O ciclo de entrega
3. Artefato: construa uma vez
4. CI, entrega e implantação contínuas
5. Ambientes
6. Integração contínua: uma introdução
7. Onde hospedar: IaaS, PaaS e container
8. Estratégias de deploy
9. Configuração e segredos na plataforma
10. Compatibilidade e breaking changes
11. Versionamento semântico
12. Formas de versionar uma API
13. Política de depreciação
14. Aplicação no ServiceHub
15. Construindo com IA
16. Glossário
17. Exercícios
18. Para estudar

1. Objetivos da semana

↑ conteúdo

Ao final desta semana você deve ser capaz de:

1. descrever o ciclo de entrega (commit, build, teste, artefato, deploy) e explicar por que o mesmo artefato deve ser promovido entre ambientes;
2. diferenciar integração contínua, entrega contínua e implantação contínua, e o papel de cada ambiente (desenvolvimento, homologação, produção);
3. ler um pipeline mínimo de integração contínua e dizer quando, onde e o que ele roda;
4. comparar IaaS, PaaS e container, e as estratégias de deploy recreate, rolling e blue-green;
5. classificar mudanças de contrato em compatíveis ou incompatíveis, aplicar versionamento semântico e propor uma política de depreciação.

2. O ciclo de entrega

↑ conteúdo

Entrega é o caminho que uma mudança faz do código do desenvolvedor até o usuário. Qualquer que seja a linguagem ou a empresa, esse caminho tem sempre as mesmas etapas. Elas podem ser feitas à mão, por uma pessoa seguindo uma lista, ou por uma máquina. Quando uma máquina executa a sequência, chamamos essa sequência de **pipeline** (esteira).



As cinco etapas de uma entrega. Automatizadas, formam o pipeline.

Exemplos fora do ServiceHub: num projeto Node, o build gera um diretório `dist/`; num projeto Java, um `.jar`. O teste pode ser unitário ou de integração, desde que rode sem intervenção humana. O artefato aprovado é guardado com um identificador único, e o deploy é a plataforma trocar a versão antiga pela nova.

A regra do pipeline é simples: **cada etapa só recebe o que a anterior aprovou**. Se o teste falha, nada é empacotado nem publicado, e o problema aparece minutos depois do commit, quando o autor ainda lembra o que mudou e corrigir é barato. Sem essa regra, o defeito é descoberto dias depois, por um usuário.

DEPLOY NÃO É RELEASE

Deploy (implantação) é colocar uma versão para rodar num ambiente. **Release** (lançamento) é torná-la disponível para os usuários. Às vezes acontecem no mesmo instante; às vezes a versão nova já está implantada, mas uma funcionalidade só é liberada depois, para um grupo ou numa data combinada.

3. Artefato: construa uma vez, implante muitas

↑ conteúdo

Artefato é o resultado empacotado do build: um `.jar`, uma imagem Docker, um pacote instalável. A propriedade mais importante de um artefato é ser **imutável**: depois de criado, ele não muda. O que muda de um ambiente para outro é apenas a **configuração** (endereço do banco, senhas, portas), que chega de fora, por variáveis de ambiente.

Promover o mesmo artefato

Uma única imagem é construída, testada, aprovada em homologação e então levada para produção. O que foi testado é exatamente o que roda. Se algo der errado, você sabe qual commit gerou aquela imagem.

Recompilar por ambiente

Um build para homologação, outro para produção, cada um com configurações embutidas. Uma dependência pode ter mudado entre os dois builds: o que roda em produção nunca foi testado de verdade.

ANALOGIA

Um lote de remédio é numerado, rastreável e igual em todas as farmácias. Ninguém fabrica um lote diferente para cada cidade; o que muda é a prateleira em que ele é colocado. O artefato é o lote; o ambiente é a prateleira.

Para rastrear, o artefato precisa de um identificador que aponte para o código de origem. O mais usado é o **SHA do commit**, o identificador único que o Git calcula para cada commit. Uma imagem chamada `minha-api:3f9c2e1` diz, sem ambiguidade, de qual commit ela saiu.

Essa regra explica uma decisão das Semanas 11 e 12: a configuração do ServiceHub fica em variáveis de ambiente, nunca no código. Se a senha do banco estivesse dentro do `.jar`, cada ambiente exigiria um build diferente, e o princípio "construa uma vez" cairia.

4. Integração, entrega e implantação contínuas

↑ conteúdo

Três práticas aparecem com nomes parecidos e uma sigla compartilhada. Cada uma inclui a anterior e acrescenta um passo de automação.

Prática	O que garante	Quem publica	Pré-requisito
Integração contínua (CI, *continuous integration*)	Cada mudança é integrada à branch principal com build e testes automáticos. O foco é manter o código integrado sempre funcionando.	Ninguém: CI não publica.	Testes automáticos confiáveis.
Entrega contínua (*continuous delivery*)	Todo commit aprovado gera um artefato pronto para produção, que pode ser publicado a qualquer momento.	Uma pessoa, com um clique.	CI e deploy automatizado.
Implantação contínua (*continuous deployment*)	Todo commit aprovado vai para produção sozinho, sem intervenção humana.	O próprio pipeline.	Entrega contínua, testes e monitoramento maduros.

CONFUSÃO COMUM

A sigla **CD** serve tanto para entrega contínua quanto para implantação contínua. A diferença entre as duas é uma só: *quem aperta o botão final*. Na entrega contínua, uma pessoa decide quando publicar; na implantação contínua, o pipeline decide. E não existe implantação contínua sem integração contínua: publicar sozinho o que nunca foi testado é só automatizar o risco.

Qual escolher depende do custo de um erro e da confiança nos testes. Uma API de pagamentos pode preferir entrega contínua, com uma pessoa aprovando o horário da publicação. Um site de conteúdo com boa cobertura de testes pode publicar sozinho a cada commit. Não há resposta única.

5. Ambientes: desenvolvimento, homologação e produção

↑ conteúdo

Ambiente é um conjunto de máquinas, banco e configuração em que uma versão roda com um propósito. Os três clássicos:

Ambiente	Para que serve	Quem usa	Dados
Desenvolvimento	Escrever, depurar e experimentar.	O desenvolvedor.	Fictícios e descartáveis.
Homologação (*staging*)	Validar a versão candidata num ambiente igual ao de produção, antes de liberá-la.	Equipe, testadores, área de negócio.	Parecidos com os reais, sem dados pessoais expostos.
Produção	Atender os usuários de verdade.	Usuários finais.	Reais, com backup.

Homologar é aprovar formalmente: em muitas empresas, o cliente ou a área de negócio testa a versão em homologação e dá o aceite antes da publicação. Para esse aceite valer, homologação precisa ser parecida com produção (mesmo banco, mesma versão do runtime, mesma imagem). Uma homologação com H2 em memória e produção com PostgreSQL não prova nada sobre produção. Essa é a **paridade entre ambientes** vista na Semana 12.

> **DADOS REAIS FORA DE PRODUÇÃO**
> Copiar a base de produção para homologação "para testar com dados de verdade" expõe dados pessoais de clientes a quem não deveria vê-los. Além de técnico, é um problema de privacidade e de conformidade com a LGPD. Use dados fictícios ou anonimizados.

No semestre, os papéis ficam assim: o Compose na sua máquina é o ambiente de desenvolvimento; o runner do CI, que sobe um PostgreSQL limpo e roda todos os testes, cumpre um papel parecido com o de homologação; a plataforma de hospedagem é a produção.

6. Integração contínua: uma introdução

↑ conteúdo

Por que existe

Quando cada pessoa trabalha semanas no próprio branch, o momento de juntar tudo vira um evento doloroso: conflitos, testes que passavam isoladamente e falham juntos, ninguém sabe qual mudança quebrou o quê. A integração contínua ataca o problema pela frequência: integrar *cedo e sempre*, em pedaços pequenos, e deixar que uma **máquina limpa** prove cada integração.

"Funciona na minha máquina" não é prova. Na sua máquina há um banco que você criou à mão, uma variável que esqueceu de documentar, uma versão do Java diferente da do colega. O servidor de CI clona o repositório do zero, baixa as dependências declaradas, sobe o que o teste pede e roda tudo. Se falhar ali, falharia em qualquer lugar.

As práticas básicas

- **Integre cedo e sempre:** branches curtos, mesclados na principal em horas ou poucos dias. Quanto maior a divergência, pior o merge.
- **Build automatizado:** um único comando constrói tudo do zero, sem passo manual.
- **Testes dentro do build:** o build só passa se os testes passam. Sem testes, o CI apenas prova que o código compila.
- **Build quebrado é prioridade:** principal vermelha para tudo; quem quebrou conserta ou desfaz a mudança antes de seguir.

A ferramenta é o menos importante: sem essas práticas, um servidor de CI só gera relatórios que ninguém lê. Estratégias de branch, pipelines com vários estágios e ambientes dentro do pipeline

são assunto da disciplina do 7º período.

Um pipeline mínimo

Existem vários serviços de CI. No GitHub, o serviço integrado se chama **GitHub Actions**: um arquivo YAML dentro do repositório, em `.github/workflows/`, descreve o que roda e quando. Como o arquivo está no repositório, o próprio pipeline é código: versionado, revisado em pull request, com histórico. Para esta disciplina, basta saber *ler* um pipeline mínimo respondendo a três perguntas:

```
on:
  push: { branches: [main] }
  pull_request: { branches: [main] }
jobs:
  build-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5
      - uses: actions/setup-java@v5
        with:
          distribution: temurin
          java-version: '21'
          cache: maven
      - run: ./mvnw -B verify
```

Pergunta	Onde está no arquivo	Resposta
Quando?	`on:`	A cada push na `main` e a cada pull request contra a `main`.
Onde?	`runs-on:`	Numa máquina Linux limpa fornecida pelo GitHub (o **runner**), que já tem Docker.
O quê?	`steps:`	Baixa o código, instala o Java 21 (distribuição Temurin, com cache das dependências Maven) e roda `./mvnw -B verify`, que compila e executa todos os testes. `-B` é o modo não interativo do Maven.

O resultado aparece no pull request como um **check**: verde se o comando terminou com sucesso, vermelho se algum teste falhou. A equipe pode configurar o repositório para não aceitar merge enquanto o check estiver vermelho.

UM CI QUE NUNCA FICOU VERMELHO NÃO FOI PROVADO

Se o pipeline nunca falhou, você não sabe se ele roda os testes de verdade. Ele pode estar pulando testes (`-DskipTests`), rodando o comando errado ou ignorando o código de saída. A prova é quebrar um teste de propósito e ver o check ficar vermelho.

7. Onde hospedar: IaaS, PaaS e container

↑ conteúdo

Depois de aprovado, o artefato precisa rodar em algum lugar acessível pela internet. A pergunta central é: **quem cuida de quê?**

Modelo	Você cuida de	O provedor cuida de	Na prática
IaaS (infraestrutura como serviço)	Sistema operacional, runtime, atualizações de segurança, rede da aplicação, backup.	Hardware, rede física e virtualização.	Uma máquina virtual na nuvem.
PaaS (plataforma como serviço)	Código e configuração.	Servidores, sistema, runtime, build, certificados e escala básica.	Ligar o repositório e publicar.
Container	A imagem: aplicação mais runtime.	Depende de onde a imagem roda.	O mesmo Dockerfile em VM ou PaaS.

Runtime é o que a aplicação precisa para rodar; numa API Java, a JRE. A regra geral: quanto mais o provedor cuida, menos trabalho você tem e menos controle também. Numa IaaS você escolhe o kernel, abre portas no firewall e aplica patches; numa PaaS você nem vê a máquina. Na analogia comum, IaaS é alugar um terreno e construir; PaaS é alugar um apartamento mobiliado. (O SaaS, software pronto como serviço, não é tema desta disciplina.)

CONFUSÃO COMUM

Container não é um terceiro modelo de hospedagem ao lado de IaaS e PaaS. É um *formato de empacotamento*. Uma imagem Docker roda numa máquina virtual (IaaS) ou numa plataforma que aceita Dockerfile (PaaS). O Dockerfile multi-stage é, por isso, o contrato de deploy da API: ele roda igual em qualquer uma das opções.

Nesta disciplina usamos PaaS, porque o foco é a API e não a administração de servidores. Planos, limites e camadas gratuitas das plataformas mudam com frequência: consulte o site oficial antes de escolher e registre a escolha num ADR curto.

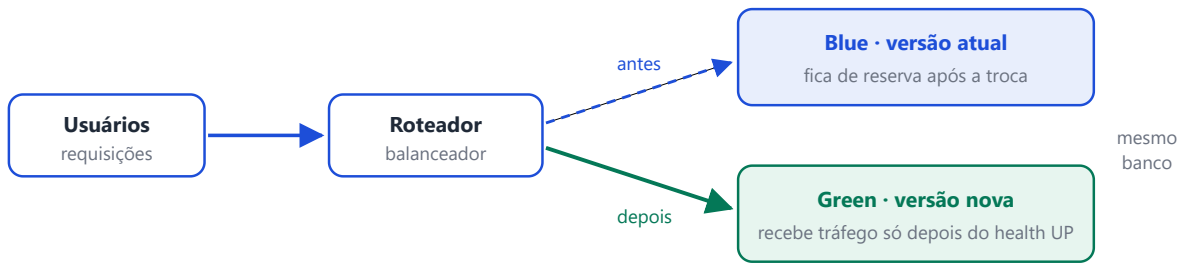
8. Estratégias de deploy

↑ conteúdo

Estratégia de deploy é a forma de trocar a versão em execução pela nova. Cada uma equilibra simplicidade, tempo fora do ar e custo. Nos exemplos, **instância** é uma cópia da aplicação em execução; uma API com tráfego costuma ter várias, atrás de um balanceador de carga.

Estratégia	Como funciona	Fica fora do ar?	Custo e volta atrás
Recreate	Para todas as instâncias antigas e depois sobe as novas.	Sim, entre parar e subir.	Simples; voltar é refazer o deploy anterior.

Estratégia	Como funciona	Fica fora do ar?	Custo e volta atrás
Rolling	Troca as instâncias aos poucos, uma ou algumas por vez.	Não, se houver mais de uma instância.	Duas versões convivem durante a troca.
Blue-green	Sobe o ambiente novo inteiro ao lado do antigo e troca o tráfego de uma vez.	Não.	Recursos em dobro durante a troca; voltar é devolver o tráfego.



Blue-green: a versão nova sobe ao lado da atual; o roteador só troca o tráfego quando ela responde ao health check.

No blue-green, a blue fica parada por um tempo depois da troca: se aparecer problema, o tráfego volta para ela. Plataformas gerenciadas costumam aplicar a mesma ideia, só desviando o tráfego para a versão nova depois que ela responde ao health check. Há ainda o **canary**, em que a versão nova recebe primeiro só uma parte pequena do tráfego; fica como curiosidade.

PONTO DELICADO: O BANCO

Rolling e blue-green só são seguros se a versão nova for compatível com o banco e com os clientes da versão antiga, porque as duas convivem por um tempo. Blue e green normalmente usam o mesmo banco: se a migração da green remove uma coluna que a blue ainda lê, a blue quebra, e a volta atrás deixa de existir. Por isso remoções de coluna se fazem em duas etapas: primeiro uma versão que para de usar a coluna; depois, numa entrega seguinte, a migração que a remove.

9. Configuração e segredos na plataforma

↑ conteúdo

O *Twelve-Factor App*, um conjunto de recomendações para aplicações em nuvem, tem como fator III: **guardar a configuração no ambiente**. Tudo o que muda entre ambientes (endereço do banco, credenciais, portas, chaves) vem de variáveis de ambiente; o código e o artefato são os mesmos. Um teste rápido: se você precisa recompilar para trocar a senha do banco, a configuração está no lugar errado.

Um **segredo** é uma configuração que dá acesso a algo: senha de banco, chave de assinatura de tokens, credencial de serviço externo. Segredos têm regras próprias:

- **Nunca no Git.** Um segredo commitado continua no histórico mesmo depois de apagado do arquivo. Se isso acontecer, troque o segredo.
- **No painel da plataforma** ou num cofre de segredos (no GitHub, os *Secrets* do repositório). A plataforma injeta o valor como variável de ambiente no container.
- **Um por ambiente.** Produção não usa a mesma chave de desenvolvimento, nem a senha padrão de exemplo.
- **Falhar cedo.** Se um segredo obrigatório faltar, a aplicação deve se recusar a subir com uma mensagem clara. Um erro no deploy é melhor que uma API insegura.

Na prática, o arquivo de configuração versionado só diz *de onde* vem o valor, por exemplo `jwt.secret=${JWT_SECRET:}`, e nunca o valor real, como `jwt.secret=minha-chave-123`. A seção 14 mostra como o ServiceHub faz isso.

FORMATO DA URL DO BANCO

Muitas plataformas entregam a URL do Postgres no formato `postgres://usuario:senha@host/banco`. O driver JDBC do Java espera `jdbc:postgresql://host:porta/banco`, com usuário e senha em variáveis separadas. Copiar a URL da plataforma sem converter é uma falha comum no primeiro deploy.

10. Compatibilidade e breaking changes

↑ conteúdo

Enquanto a API roda só na sua máquina, você pode renomear um campo e ajustar o front no mesmo commit. Depois do deploy, há clientes que você não controla: um aplicativo instalado no celular de alguém, o sistema de um parceiro, o front de outra equipe. Mudar o contrato passa a ter regra.

Breaking change (mudança incompatível) é qualquer mudança que obriga um cliente existente a mudar para continuar funcionando. **Mudança compatível** (*non-breaking*) é a que nenhum cliente percebe como quebra. A garantia que uma API pública precisa dar é a **compatibilidade com o passado** (*backward compatibility*): um cliente feito para a versão antiga continua funcionando com o servidor novo.

Compatibilidade é responsabilidade dos dois lados. O servidor não remove nem muda o que existe. O cliente segue o princípio do **leitor tolerante**: ignora campos que não conhece e não depende da ordem deles. Com essas duas atitudes, acrescentar um campo não quebra ninguém.

Mudança	Exemplo numa API de pedidos	Efeito
Remover ou renomear campo	`total` passa a se chamar `amount`	Quebra
Mudar o tipo de um campo	`total` passa de número para texto	Quebra

Mudança	Exemplo numa API de pedidos	Efeito
Novo campo obrigatório na entrada	`POST /orders` passa a exigir `deliveryDate`	Quebra
Mudar status code ou regra	`POST` devolve 200 em vez de 201; um campo opcional passa a ser validado	Quebra
Novo valor de enum na resposta	Um status novo, `ON_HOLD`	Pode quebrar
Novo campo na resposta	Incluir `customerName` no JSON	Compatível
Novo campo opcional na entrada	`POST /orders` aceita `notes`, sem exigir	Compatível
Novo endpoint	`GET /orders/{id}/history`	Compatível

Um valor novo de enum quebra o cliente que faz `switch` sem caso padrão, ou que valida a resposta contra uma lista fixa. E "novo campo é compatível" só vale se o cliente ignora campos desconhecidos. Por isso essas expectativas devem estar escritas no contrato.

11. Versionamento semântico

↑ conteúdo

O **versionamento semântico** (*Semantic Versioning*, descrito em `semver.org`) dá significado a cada número de uma versão no formato `MAJOR.MINOR.PATCH`:

Número	Exemplo	Quando sobe
MAJOR	`1.9.0 → 2.0.0`	Mudança incompatível: algum cliente vai quebrar se atualizar sem ajustar.
MINOR	`2.3.0 → 2.4.0`	Funcionalidade nova, compatível com o que existia.
PATCH	`2.4.0 → 2.4.1`	Correção de defeito, compatível.
0.y.z	`0.3.1`	Desenvolvimento inicial: pela especificação, tudo pode mudar a qualquer momento.

Quando um número sobe, os da direita voltam a zero: depois de `2.4.1`, uma funcionalidade nova gera `2.5.0`, e uma quebra gera `3.0.0`. Quem lê a versão sabe, sem abrir o changelog, se pode atualizar com segurança.

Numa API web, o cliente só precisa enxergar o **MAJOR**: é ele que vira o `v1` da URL. MINOR e PATCH ficam na versão do artefato, como a tag Git `v1.4.0` que marca um release e a tag da imagem. Por isso a versão da API não precisa acompanhar a versão do `pom.xml`: o número da URL só muda quando o contrato quebra.

As tags que você cria no repositório (`etapa-1`, `etapa-2`, … `etapa-final`) marcam entregas da disciplina, não versões semânticas. Numa API real, cada release teria uma tag no formato semântico.

12. Formas de versionar uma API

↑ conteúdo

Quando uma mudança incompatível é inevitável, a versão antiga e a nova precisam coexistir por um tempo, e o cliente precisa dizer qual quer. Há três formas comuns de dizer a versão:

Onde	Exemplo	Prós e contras
Na URI	`GET /api/v1/products` `GET /api/v2/products`	Explícita, testável no navegador e no `curl`, fácil de rotear e de documentar. Repete o prefixo em toda rota.
Num header próprio	`GET /api/products` `Accept-Version: 2`	URL estável, mas a versão fica escondida: não dá para testar só com a barra do navegador. O header é uma convenção da API, não um padrão HTTP.
No media type	`GET /api/products` `Accept: application/vnd.loja.v2+json`	Usa a negociação de conteúdo do HTTP; é o mais fiel ao protocolo e o mais trabalhoso de usar e documentar. O prefixo `vnd.` indica um tipo específico de fornecedor (*vendor*); "loja" é só um exemplo.

Escolha uma estratégia e mantenha. Em qualquer uma delas, a versão só muda quando o contrato quebra; acrescentar um campo ou um endpoint entra na versão atual. O ServiceHub versiona pela URI: todas as rotas vivem sob `/api/v1` desde o primeiro endpoint. A escolha foi pela simplicidade e porque a bancada de testes do professor chama rotas fixas. O `v1` é uma promessa: enquanto ele existir, quem o usa não quebra.

13. Política de depreciação

↑ conteúdo

Depreciar é anunciar que algo vai deixar de existir, com prazo, antes de remover. A **política de depreciação** é a regra pública que diz como e com quanta antecedência isso acontece. Sem política, cada remoção vira uma surpresa para quem consome a API.

1. **Anunciar.** Changelog, documentação e aviso direto a quem consome a API, com a data de desligamento.
2. **Marcar.** No OpenAPI, a operação recebe `deprecated: true`, e o Swagger UI a mostra riscada. Nas respostas, cabeçalhos avisam o cliente em tempo de execução (veja abaixo).

3. **Conviver.** `v1` e `v2` ficam no ar pelo prazo anunciado. Os logs mostram quem ainda chama a versão antiga, para avisar diretamente.
4. **Remover.** Só depois do prazo. A rota antiga pode passar a responder `410 Gone`, que significa "existiu e foi removido de propósito", diferente de `404 Not Found`.

Avisar na própria resposta

A **RFC 8594** define o cabeçalho `Sunset`, que informa a data e hora a partir da qual o recurso deve deixar de responder. A data segue o formato de datas do HTTP. Um exemplo, com data fictícia:

```
HTTP/1.1 200 OK
Content-Type: application/json
Sunset: Wed, 30 Jun 2027 23:59:59 GMT
```

Além da data de desligamento, existe a ideia de **marcar a resposta como depreciada**, para que o cliente saiba que deve migrar mesmo antes de haver data. Um cabeçalho chamado `Deprecation` é usado com esse propósito; nesta disciplina basta conhecer o conceito: depreciação diz "não use mais", sunset diz "a partir de quando deixa de existir".

O prazo depende de quem consome: parceiros externos costumam precisar de meses para planejar a migração; o front da própria equipe pode migrar em dias. A política deve dizer o prazo mínimo e onde os avisos são publicados.

14. Aplicação no ServiceHub

↑ conteúdo

O pipeline do projeto

O arquivo `.github/workflows/ci.yml` do ServiceHub tem dois jobs. O primeiro, `build-test`, é o pipeline mínimo da seção 6 com nomes nos passos (por exemplo, `name: Java 21 (Temurin)`) e o upload dos relatórios de cobertura e de teste. O segundo só existe para provar a imagem. Trecho real, resumido:

```
  docker:
    name: Imagem Docker
    runs-on: ubuntu-latest
    needs: build-test
    steps:
      - uses: actions/checkout@v5
      - name: docker build
        run: docker build -t servicehub-api:${{ github.sha }} .
      # (resumido: sobe o Compose, espera o container ficar healthy
      #  num laço com limite de tentativas e chama /api/v1/info)
```

Três detalhes ligam o pipeline à teoria. Os testes usam PostgreSQL real via Testcontainers, não H2 (ADR-0005): o runner do GitHub já tem Docker, então o CI prova contra o mesmo banco da

produção. O segundo job só roda se o primeiro passar (`needs: build-test`), aplicando a regra "cada etapa só recebe o que a anterior aprovou". E a imagem recebe como tag o SHA do commit, o identificador de artefato da seção 3.

> **ERRO COMUM: MVNW SEM PERMISSÃO**
> Quando o `mvnw` é commitado a partir do Windows, ele pode perder o bit de execução, e o runner Linux recusa `./mvnw` com "Permission denied". A correção é `git update-index --chmod=+x mvnw`. No projeto, `git ls-files -s mvnw` mostra o modo `100755`.

Um único sinal de saúde

O mesmo endpoint atravessa toda a cadeia de entrega. O Actuator responde em `/actuator/health`; o `Dockerfile` usa a rota no `HEALTHCHECK`; o CI espera o container ficar *healthy*; e na plataforma de deploy você informa o mesmo caminho como health check, para que ela só troque a versão quando a nova responder. Trecho real do `Dockerfile`:

```
HEALTHCHECK --interval=15s --timeout=5s --start-period=40s --retries=5 \
  CMD wget -qO- http://localhost:8080/actuator/health | grep -q '"status":"UP"' || exit 1
```

Por isso a rota precisa ser pública. Em `security/SecurityConfig`:

```
.requestMatchers("/actuator/health", "/actuator/health/**", "/actuator/info").permitAll()
.requestMatchers("/actuator/**").hasRole("ADMIN")
```

Se o health ficasse atrás de autenticação, o health check receberia 401 e o container nunca ficaria saudável.

As variáveis que a API lê

O `application.properties` segue o fator III: cada valor que muda por ambiente vem de uma variável, com um padrão de desenvolvimento depois dos dois-pontos. Trecho real, resumido:

```
server.port=${SERVER_PORT:8080}
spring.datasource.url=${DB_URL:jdbc:postgresql://localhost:5433/servicehub}
spring.datasource.password=${DB_PASS:secret}
servicehub.jwt.secret=${JWT_SECRET:}
```

Em produção: `SPRING_PROFILES_ACTIVE=prod` (liga os logs em JSON, tema da Semana 15); `DB_URL`, `DB_USER` e `DB_PASS` do Postgres gerenciado, em formato JDBC; `JWT_SECRET` novo, com 32 bytes ou mais; `SERVICEHUB_ADMIN_EMAIL` e `SERVICEHUB_ADMIN_PASSWORD` de um admin real, nunca a senha de exemplo; `CORS_ALLOWED_ORIGINS` só com a URL do front publicado; e `SERVER_PORT`, se a plataforma indicar uma porta.

A lista completa, com valores de exemplo, está no `.env.example` versionado; o `.env` real é ignorado pelo Git. O "falhar cedo" está em `security/JwtProperties`: o construtor do record lança `IllegalStateException` com a mensagem "Defina JWT_SECRET com pelo menos 32 bytes..." se o segredo estiver ausente ou curto, e a API não sobe.

Publicar numa PaaS, em quatro passos

(1) Conecte o repositório e escolha a branch `main`; (2) aponte o build para o `Dockerfile` da raiz, o mesmo que o CI constrói; (3) crie o Postgres gerenciado na plataforma (o Flyway aplica as migrações no primeiro start); (4) preencha as variáveis listadas acima e informe `/actuator/health` como health check. É no passo 4 que quase todo primeiro deploy falha. Está pronto quando `GET /api/v1/info` responde na URL pública e a bancada de testes completa o ciclo contra ela. Os nomes dos menus mudam de plataforma para plataforma; siga a documentação oficial da escolhida.

O contrato v1

Todo controller do projeto declara o prefixo, por exemplo `@RequestMapping("/api/v1/services")` em `ServiceOfferingController` e `@RequestMapping("/api/v1/service-requests")` em `ServiceRequestController`. O `OpenApiConfig` publica esse contrato num grupo próprio, com `.group("v1")` e `.pathsToMatch("/api/v1/**")`; a especificação fica em `/api-docs/v1`. Aplicando a seção 10 ao `ServiceRequestResponseDTO` real: acrescentar um campo como `providerName` é compatível e entra na v1; renomear `totalPrice` para `price` ou mudar seu tipo é breaking e pediria uma v2.

15. Construindo com IA

↑ conteúdo

A IA vai escrever um arquivo pequeno, mas que decide se as mudanças futuras serão provadas. Um pipeline errado pode ficar verde por muito tempo porque não roda nada. Os quatro movimentos, aplicados ao CI:

Movimento	Nesta semana
Especificar	A cada push e pull request na `main`, o pipeline roda `./mvnw -B verify` com Java 21 e PostgreSQL real e falha se um teste falhar.
Gerar	Peça o plano (gatilhos, máquina, passos) antes do YAML, e o YAML mínimo, sem jobs extras.
Revisar	Versões das actions, gatilhos, nenhum segredo no arquivo, nada de H2 nem de `-DskipTests` para "acelerar".
Provar	Quebre um teste de propósito num branch e abra um PR: o check tem de ficar vermelho. Corrija e veja voltar ao verde.

Prompt fraco e prompt bom

Fraco

```
cria um CI/CD pro meu projeto spring
```

Sem versão do Java, comando de build nem escopo, volta um pipeline longo, com deploy, segredos inventados e actions antigas.

Bom

Diz o projeto, o escopo (só CI), os gatilhos, as versões das actions e o que é proibido; pede o plano antes e define como provar que o pipeline falha quando deve. É o prompt abaixo.

O prompt desta semana

Continue de onde a Semana 12 parou (Dockerfile e Compose funcionando). Cole no assistente e ajuste ao seu repositório, se algo for diferente:

```
Contexto: somos uma equipe construindo a API ServiceHub em Java 21 e
Spring Boot 4.1, com Maven Wrapper (mvnw na raiz), PostgreSQL 16,
migrações Flyway e testes de integração com Testcontainers 2
(postgres:16-alpine). O repositório está no GitHub, branch main.
Hoje os testes rodam localmente com: ./mvnw verify (Docker ligado).

Objetivo: criar um pipeline MÍNIMO de integração contínua no GitHub
Actions que compile e rode todos os testes a cada push na main e a
cada pull request contra a main.
```

```
Restrições:
- Arquivo único: .github/workflows/ci.yml, com um único job.
- Runner ubuntu-latest (já tem Docker, que o Testcontainers usa).
- Usar exatamente actions/checkout@v5 e actions/setup-java@v5 com
  distribution: temurin, java-version: '21' e cache: maven.
- O único comando de build é: ./mvnw -B verify
- NÃO usar H2 nem trocar o banco dos testes; NÃO usar -DskipTests,
  -Dmaven.test.failure.ignore nem continue-on-error.
- NÃO colocar segredo nem JWT_SECRET no YAML; NÃO incluir deploy.
- Se o mvnw não tiver permissão de execução, me diga o comando git
  para corrigir, em vez de trocar por "mvn".

Antes do código: escreva um plano curto respondendo "quando roda,
onde roda, o que roda" e liste o que pode fazer o pipeline ficar
verde sem testar nada. Espere minha confirmação.

Critério de aceite: o pull request mostra o check verde e um teste
quebrado deixa o check vermelho.
```

O que a IA costuma errar nesta etapa

- **Actions antigas:** `checkout@v3` e `setup-java@v3`, copiadas de tutoriais antigos. O projeto usa `@v5` nas duas.
- **Trocar o banco dos testes** por H2 "para ficar mais rápido", o que quebra as migrações escritas para o PostgreSQL.
- **Pular os testes** com `-DskipTests` ou marcar o passo com `continue-on-error: true`: o check fica sempre verde.
- **Segredo no YAML** em vez de GitHub Secrets; **pipeline completo**, com deploy, quando só o CI foi pedido; trocar `./mvnw` por `mvn` quando o wrapper falha por permissão.

Checklist de revisão

☐ Os gatilhos são `push` e `pull_request`, ambos na `main`?

☐ As actions estão em `@v5`, com Temurin e Java `'21'` entre aspas?

☐ O comando é `./mvnw -B verify`, sem nenhuma opção que pule ou ignore testes?

☐ `git ls-files -s mvnw` mostra o modo `100755`?

☐ Eu sei explicar, linha por linha, o que cada parte do YAML faz?

Prova

Crie um branch, altere a expectativa de um teste para um valor errado (por exemplo, esperar 200 onde a API devolve 201), faça push e abra um pull request contra a `main`. O check precisa ficar **vermelho**, e o log do job precisa mostrar o nome do teste que falhou. Desfaça a alteração, faça push no mesmo branch: o check volta ao **verde**. Guarde o link das duas execuções na aba Actions como evidência da entrega.

16. Glossário

↑ conteúdo

Pipeline	Sequência automatizada de etapas que leva uma mudança do commit ao deploy.
Artefato	Resultado imutável e versionado do build, como um `.jar` ou uma imagem Docker.
Deploy	Implantação: colocar uma versão para rodar num ambiente.
Release	Lançamento: tornar uma versão disponível para os usuários.
Integração contínua (CI)	Integrar cada mudança à branch principal com build e testes automáticos.
Entrega contínua	Todo commit aprovado gera um artefato pronto para produção; uma pessoa decide quando publicar.
Implantação contínua	Todo commit aprovado é publicado em produção pelo próprio pipeline.
Homologação	Ambiente igual ao de produção, usado para validar e aprovar a versão candidata.
Runner	Máquina que executa os jobs de um pipeline.
IaaS / PaaS	Infraestrutura como serviço (você administra a máquina) / plataforma como serviço (você entrega código e configuração).
Blue-green	Estratégia de deploy em que a versão nova sobe ao lado da atual e recebe o tráfego de uma vez.
Breaking change	Mudança que obriga um cliente existente a mudar para continuar funcionando.
Semver	Versionamento semântico: `MAJOR.MINOR.PATCH`, com significado para cada número.
Sunset	Cabeçalho HTTP (RFC 8594) que informa quando um recurso deixará de responder.

17. Exercícios

↑ conteúdo

1. Explique, com suas palavras, a diferença entre entrega contínua e implantação contínua. Para uma API de pagamentos, qual você escolheria e por quê?
2. Uma equipe gera um `.jar` para homologação e, depois do aceite, faz outro build para produção com o arquivo de configuração de produção embutido. Que princípio foi quebrado? Que risco isso traz?
3. Leia o pipeline mínimo da seção 6 e responda: quando ele roda, onde roda e o que roda. O que acontece com o pull request se um teste de integração falhar?
4. Um colega diz: "nosso CI está ótimo, nunca ficou vermelho". Que perguntas você faria para saber se o pipeline realmente roda os testes?

5. Classifique como IaaS, PaaS ou nenhum dos dois, justificando: (a) uma máquina virtual em que você instala o Java; (b) uma plataforma que lê o seu repositório e publica pelo Dockerfile; (c) uma imagem Docker.

6. A versão nova de uma API remove a coluna `phone` da tabela de clientes, e o deploy é blue-green com banco compartilhado. O que pode dar errado durante a troca? Como dividir a mudança para torná-la segura?

7. Classifique cada mudança no `ServiceRequestResponseDTO` como compatível, incompatível ou "pode quebrar": (a) acrescentar `providerName`; (b) mudar `totalPrice` de número para texto; (c) acrescentar um sexto valor ao enum `RequestStatus`; (d) renomear `scheduledAt` para `scheduledFor`.

8. Uma biblioteca está na versão `3.7.2`. Qual é a próxima versão em cada caso: (a) corrige um defeito; (b) acrescenta um método novo; (c) remove um método público?

9. Compare o versionamento pela URI e pelo media type. Por que o ServiceHub escolheu a URI?

10. Escreva uma política de depreciação de cinco linhas para a `v1` do ServiceHub, citando onde o aviso é publicado, o prazo, os cabeçalhos e o status depois da remoção.

11. A API sobe na plataforma, mas o log mostra "Defina JWT_SECRET com pelo menos 32 bytes". Onde está o problema e como resolver sem alterar o código?

12. Reescreva o prompt fraco "cria um CI/CD pro meu projeto spring" para pedir apenas o pipeline mínimo, incluindo pelo menos três restrições e o critério de aceite.

18. Para estudar

↑ conteúdo

Para a 3ª VA, priorize os conceitos: ciclo de entrega, CI e as duas CDs, ambientes, modelos de hospedagem, estratégias de deploy, compatibilidade, semver e depreciação. Da sintaxe do workflow, basta saber ler o pipeline mínimo. Leituras disponíveis na Minha Biblioteca, pelo acesso institucional:

- MONTEIRO, E. R. et al. *DevOps*. Grupo A, 2021. Capítulo "Integração e entrega contínuas" (leitura introdutória).
- PRESSMAN, R. S.; MAXIM, B. R. *Engenharia de software*. 9. ed. Grupo A, 2021. Capítulo 22.

Referências abertas:

- Semantic Versioning: `semver.org`.
- RFC 8594, *The Sunset HTTP Header Field*: `rfc-editor.org/rfc/rfc8594`.
- The Twelve-Factor App, fator III (Config): `12factor.net`.
- Documentação oficial do GitHub Actions, guia de início rápido: `docs.github.com/actions`.